

Smart Rummy: An Explainable Utility-Based Agent for 13-Card Indian Rummy Using Memoised Bitmask Dynamic Programming and Heuristic Simulation

Jobeson Julius

Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Jobeson.j.2024.cse@rajalakshmi.edu.in

Pragadeesh S

Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Pragadeesh.s.2024.cse@rajalakshmi.edu.in

Abstract—Indian Rummy is a partially observable, multi-player card game of skill that is played by millions in India, yet its digital versions offer opponents whose decisions are opaque and never explained. This paper presents Smart Rummy, an explainable game-playing system in which every computer player is a utility-based agent built only from classical artificial-intelligence techniques. Declarations are verified by depth-first backtracking with failure memoisation. A hand is evaluated by a memoised bitmask dynamic program that branches on the lowest undecided card and returns the minimum-deadwood arrangement under the declaration constraints. On this evaluation we define a progress-cost utility and choose actions by simulating every discard, by one-step lookahead on the open card and by exact probabilities over unseen cards. Three agent designs (simple reflex, utility-based and model-based) provide graded difficulty, and a coach turns the same computations into plain-language advice, outs and opponent reads for a human learner. The system is implemented with FastAPI and React and is verified by 123 automated tests. A 13-card hand is evaluated in 22.6 ms on average and a utility-based agent turn takes about 150 ms. In 100-round head-to-head matches, the utility-based agents defeat the reflex agent in 65–66% of rounds ($p < 0.01$).

Keywords—Indian Rummy, game-playing agents, imperfect information, dynamic programming, backtracking search, explainable AI, intelligent tutoring

I. INTRODUCTION

Rummy is the most popular real-money card game in India: a 2024 survey of 1,044 real-money gamers found it to be the most played game in that category [1]. Indian courts have treated rummy as a game of skill since 1967 [2], and a 2023 study by IIT Delhi of thousands of online players found that skill, not luck, determines long-run outcomes in rummy and poker [3]. A game in which skill dominates is a natural domain for teaching, and a natural benchmark for artificial intelligence.

Card games are hard for AI because they combine hidden information, chance and several self-interested players. Landmark systems such as Libratus [4] and Pluribus [5] reached superhuman strength in poker, but their strategies are computed through large-scale equilibrium search and cannot be explained to a learner. Commercial rummy applications show the same gap: they provide an opponent, but never say why a move was good. Explainable AI aims to make such decisions understandable [6], and in games it can support players and designers alike [7]. Evidence from education

shows that step-by-step intelligent tutoring can approach the effectiveness of a human tutor [8].

This paper presents **Smart Rummy**, a web-based Indian Rummy game in which every computer player is a rational agent in the sense of Russell and Norvig [9], and in which the same reasoning powers an interactive coach. Its contributions are:

- A formulation of 13-card Indian Rummy as a partially observable task environment, with a progress-cost utility that remains informative before a hand has a valid structure.
- An exact hand evaluator based on memoised bitmask dynamic programming with lowest-undecided-card branching, a constrained/unconstrained two-pass design and spare-joker placement, together with a backtracking declaration verifier.
- Three agent designs (simple reflex, utility-based and model-based) built on discard simulation, one-step lookahead and exact probability, evaluated in 300 head-to-head rounds.
- A coach that explains every recommendation, lists outs, infers opponents' plans from discard-pile pickups and grades the player's moves, deployed in a tested full-stack system.

Section II reviews related work, Section III formulates the problem, Section IV describes the proposed system, Section V gives the experimental setup, Section VI reports the results and Section VII concludes.

II. RELATED WORK

A. AI for imperfect-information card games

Determinisation samples the hidden cards and solves each perfect-information instance; Long *et al.* explain when this works despite its theoretical flaws [10]. Information Set Monte Carlo Tree Search avoids strategy fusion by searching over information sets [11], within the wider family of Monte Carlo tree search methods [12]. Equilibrium-based solvers achieved superhuman poker play [4], [5], and opponent modelling from observed actions was shown to be valuable early on in poker [13]. These methods are strong but computationally heavy, and their decisions are statistical rather than explainable.

B. Rummy-family agents

Kotnik and Kalita trained Gin Rummy agents by temporal-difference learning and by evolution in self-play [14]. The EAAI-21 Gin Rummy challenge produced several agents: a convolutional hand evaluator combined with Bayesian defensive reasoning [15], hand evaluation by meld distance with opponent modelling [16], and rule-based agents with tuned parameters, Heisenbot [17] and Tonic [18]. RLCARD provides reinforcement-learning environments for card games including Gin Rummy [19]. These works target Gin Rummy, which has neither jokers nor the pure-sequence rule.

C. Meld optimisation

Den Hertog and Hulshof solved Rummikub tile placement exactly by integer linear programming [20], and van Rijn *et al.* analysed the problem’s complexity, giving a dynamic program that is polynomial for a fixed number of suits and copies [21]. Both treat optimisation in isolation, without a playing agent.

D. Indian Rummy

Indian Rummy has only recently been studied directly. Saha *et al.* define MinScore, the minimum deadwood computed by bitmask dynamic programming, and MinDist, the number of replacements needed to declare, and report that a MinDist strategy with opponent modelling wins over 60% of 1,000-game match-ups [22]. Mohan trains IRumAI with proximal policy optimisation after a behaviour-cloning warm start, winning 53.9% against the strongest search-based baseline [23]. Our evaluator is related to MinScore, but our focus differs: a utility that penalises missing structure, a deployed interactive system, graded agent designs and, above all, explanations for a human player. Table I summarises the comparison.

TABLE I
COMPARISON WITH RELATED WORK

Work	Game	Technique	Explains moves	Live coach
[14]	Gin	TD learning / evolution	No	No
[15]	Gin	CNN + Bayesian	No	No
[17], [18]	Gin	Tuned rules	No	No
[20], [21]	Rummikub	ILP / DP	No	No
[22]	Indian	Bitmask DP metrics	No	No
[23]	Indian	PPO (deep RL)	No	No
Ours	Indian	DP + simulation	Yes	Yes

III. PROBLEM FORMULATION

A. Game rules

Indian Rummy uses two standard decks and two printed jokers (106 cards). Each player holds 13 cards. A card cut at the start of a round makes every card of its rank a *wild joker*, and a cut printed joker makes Aces wild. A *pure sequence* is three or more consecutive cards of one suit without any joker used as a substitute; an *impure sequence* uses jokers to fill gaps; a *set* is three or four cards of one rank in distinct suits. The Ace may be low (A-2-3) or high (Q-K-A), but runs may not wrap (K-A-2). On each turn a player draws from the stock or the discard pile and then discards or declares. A valid declaration groups all 13 cards with at least two sequences,

one of them pure. Losers score the points of their ungrouped cards (face cards and Aces count 10, jokers 0), capped at 80. Dropping costs 20 points before the first draw and 40 afterwards, and an invalid declaration (*wrong show*) costs 80.

B. Task environment

The environment is partially observable (the stock order and opponents’ hands are hidden), stochastic, sequential, static, discrete and competitive multi-agent [9]. Table II gives the PEAS description.

TABLE II
PEAS DESCRIPTION OF THE AGENT

Component	Description
Performance	Declare first; otherwise minimise deadwood and cumulative score
Environment	106-card shoe, stock, discard pile, wild rank, 2–4 players
Actuators	Draw (stock or discard), discard a card, declare
Sensors	Own hand, open card, discard history, opponents’ pickups

C. Hand evaluation and utility

Let a hand H contain n cards, and let M be the set of valid melds of three to five cards drawn from H . An arrangement $A \subseteq M$ is a set of pairwise disjoint melds. Its loose points are

$$L(A) = \sum_{c \in H \setminus \cup A} v(c), \quad (1)$$

where $v(c)$ is the point value of card c . A is *structurally valid* if it contains at least one pure sequence and at least two sequences. The deadwood of the hand is

$$D(H) = \min_{A \text{ valid}} L(A), \quad (2)$$

and $D(H) = \min(80, \sum_{c \in H} v(c))$ when no valid arrangement exists.

Before a hand has a valid structure, D is the full count and does not distinguish promising hands from poor ones. We therefore use a *progress cost* as the agent’s (negative) utility. Let A^* be the arrangement that maximises the points grouped, and let p and s be the numbers of pure and total sequences in it:

$$C(H) = L(A^*) + 30 \cdot [p = 0] + 15 \cdot [s < 2], \quad (3)$$

and $C(H) = D(H)$ once the hand is structurally valid. The weights 30 and 15 reflect that the pure sequence is the harder and mandatory requirement.

IV. PROPOSED SYSTEM

A. Architecture

Smart Rummy has a layered client–server architecture (Fig. 1). A React and TypeScript client renders the table, the hand and the coach panel and paces bot turns. A FastAPI service exposes 13 REST endpoints with Pydantic validation. The game engine maintains the turn state machine, scoring and the agents, and never reveals bot hands to the client. All AI components are pure functions over card lists, so the agents, the hints and the tests share the same code. The shoe is shuffled with a seeded Fisher–Yates shuffle [24], so any game can be replayed exactly.

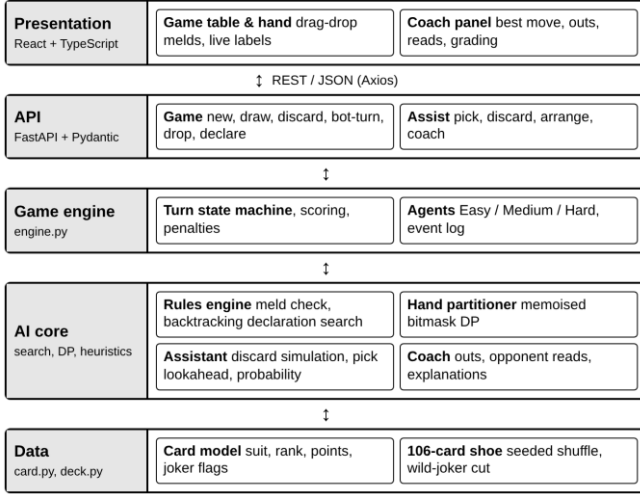


Fig. 1. Layered architecture of Smart Rummy.

B. Declaration verification by backtracking

Whether a hand can be declared is a constraint-satisfaction problem. Algorithm 1 fixes the first remaining card, tries every valid group of 3–8 cards containing it, and memoises remainders that are known to be unsolvable. Fixing the first card avoids generating the same partition in several orders.

Algorithm 1: Declaration search

```

1 function Search( $R, G$ )
2   if  $R = \emptyset$  then return  $G$  if  $\#seq \geq 2$  and  $\#pure \geq 1$  else FAIL
3   if  $R \in Dead$  then return FAIL
4    $f \leftarrow$  first card of  $R$ 
5   for each valid group  $g \subseteq R, f \in g, 3 \leq |g| \leq 8$  do
6      $r \leftarrow Search(R \setminus g, G \cup \{g\})$ 
7     if  $r \neq FAIL$  then return  $r$ 
8    $Dead \leftarrow Dead \cup \{R\}$ ; return FAIL

```

Meld validity is checked strictly. Runs are tested under both Ace interpretations. For impure sequences, natural cards act as anchors and a window of the target length must fit within legal bounds with its gaps filled by the available jokers.

C. Hand evaluation by bitmask dynamic programming

Computing $D(H)$ is a weighted set-packing problem. Algorithm 2 encodes decided cards as a bitmask and always branches on the lowest undecided card i : either i is deadwood, or it joins a candidate meld that contains it and is disjoint from the mask. Each card is decided exactly once, so the same subset is never reached by permuted choices. States $(mask, p, s)$ with p, s capped at 2 are memoised with back-links, from which the optimal melds are rebuilt. The lowest undecided card is isolated in $O(1)$ as $(\neg mask \wedge full) \wedge \neg(\neg mask \wedge full)$.

Algorithm 2: Minimum-deadwood arrangement

```

1 Enumerate valid melds of 3–5 cards; index by lowest card
2 function Go( $mask, p, s$ )
3   if  $mask = full$  then return 0 if  $(p \geq 1, s \geq 2)$  else  $-\infty$ 
4   if  $(mask, p, s) \in memo$  then return  $memo[\cdot]$ 
5    $i \leftarrow$  lowest undecided card
6    $best \leftarrow Go(mask \cup \{i\}, p, s) \triangleright i$  is deadwood
7   for each meld  $m \ni i$  with  $m \cap mask = \emptyset$  do
8      $best \leftarrow \max(best, v(m) + Go(mask \cup m, p', s'))$ 
9    $memo[(mask, p, s)] \leftarrow best$ ; return  $best$ 

```

The search runs twice. The constrained pass enforces structural validity; if it fails, an unconstrained pass returns the most useful grouping for display and the full count is used. Because a joker adds no points to a meld, the optimiser has no

incentive to place spare jokers, so a final step inserts them into existing melds to keep the hand declarable. Over 200 random hands, a 13-card hand had 27.4 candidate melds on average (median 13; jokers raise the maximum to 246).

D. Action selection

Discard. The discard decision is a one-ply search over the 14 cards held after drawing. For each card c , the remaining 13 cards are evaluated and the outcomes are ranked lexicographically by

$$key(c) = (C(H \setminus \{c\}), \kappa(c), -v(c)), \quad (4)$$

so the smallest distance to a declaration wins, ties go to the card least connected to the loose cards, and then to the higher-value card. Candidate melds are enumerated once for the 14-card hand and reused in all 14 evaluations. Discarding a joker adds a penalty of 100. The connectivity $\kappa(c)$ gives 25 points for each loose card of the same rank in another suit, or of the same suit within two ranks, capped at 100, minus 10 for each neighbour whose two copies are already in the discard pile (card counting).

Pick. The open card t is taken only if it would be kept and the hand strictly improves, which is a one-step lookahead: $\min_{c \in T} C(H \cup \{t\} \setminus \{c\}) < C(H)$ with the minimising $c \neq t$.

Probability. For a set of useful card identities T , the chance that the next stock card helps is exact given the agent’s information:

$$P = \sum_{x \in T} (2 - \text{seen}(x)) / (106 - |\text{seen}|). \quad (5)$$

E. Agent designs

The three difficulty levels are three agent designs (Fig. 2, Table III). All agents declare as soon as 13 cards are grouped with one card left for the finish slot.

TABLE III
AGENT DESIGNS

Level	Agent type	Pick	Discard
Easy	Simple reflex	Jokers only	Highest loose card
Medium	Utility-based	Lookahead	Simulation
Hard	Model-based utility	Lookahead	Simulation + dead-card memory

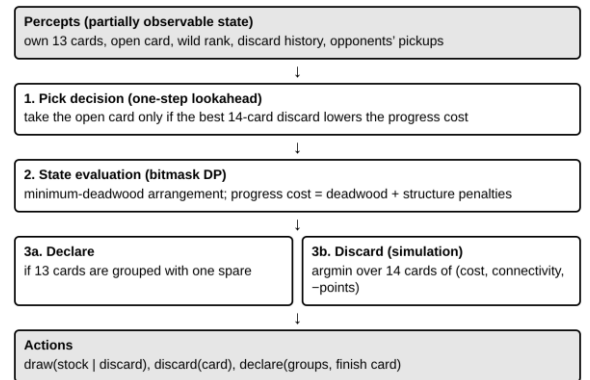


Fig. 2. Decision cycle of the utility-based agent.

F. Explainable coach

The coach reuses the agent’s computations to teach the player. It states the best move with a reason naming the concrete gain (for example, “gives you a pure sequence”). It lists *outs*, the unseen cards that would lower $C(H)$, found by adding each candidate identity to the hand as a probe card,

together with the copies left. It reads opponents from their discard-pile pickups and warns against feeding the next player; when two discards are equally good, the one adjacent to that player’s pickups is ranked second. Every human move is graded as *best*, *close* or *better option*, and a running match rate is shown (Fig. 3).



Fig. 3. Coach panel explaining the recommended discard and reviewing the previous move.

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

The back end is written in Python with FastAPI and Pydantic, and the front end in React 19 with TypeScript, Vite and Tailwind CSS. Correctness is checked by 123 Pytest cases covering the deck, the rules, every algorithm, the REST API and complete simulated games. Run time was measured on 200 seeded random hands, and agent turn times were recorded in simulated games. Playing strength was measured in head-to-head two-player rounds between every pair of agent levels: 100 seeded rounds per pairing, alternating the first player to cancel first-mover advantage. Win rates are reported with Wilson 95% confidence intervals and two-sided exact binomial tests against 50%.

VI. RESULTS AND DISCUSSION

A. Run time

Table IV reports execution times. Every operation completes far below the 950 ms pause the interface inserts between bot moves, so the game feels instant even with three opponents. A utility-based turn costs about 150 ms because it runs the partitioner once for the pick decision and 14 times for the discard.

TABLE IV
EXECUTION TIME (MS)

Operation	Mean	95th pct.
Hand partitioner, 13 cards	22.6	29.8
Discard simulation, 14 cards	39.7	61.0
Declaration search	63.0	157.2
Easy agent turn	37.5	73.4
Medium agent turn	151.2	236.9
Hard agent turn	146.9	258.6

B. Playing strength

Table V and Fig. 4 give the head-to-head results. Both utility-based agents clearly outperform the reflex agent: Medium won 65% of rounds against Easy (95% CI 55%–74%, $p = 0.0035$), and Hard won 66% (95% CI 56%–75%, $p =$

0.0018). The reflex agent also conceded about twice as many penalty points per round (24.9 vs 13.1 against Medium).

TABLE V
HEAD-TO-HEAD RESULTS (100 ROUNDS EACH)

Pairing (A vs B)	B wins	95% CI	p	Pts/round A / B
Easy vs Medium	65%	55%–74%	0.0035	24.9 / 13.1
Easy vs Hard	66%	56%–75%	0.0018	25.3 / 11.4
Medium vs Hard	50%	40%–60%	1.00	22.2 / 19.2

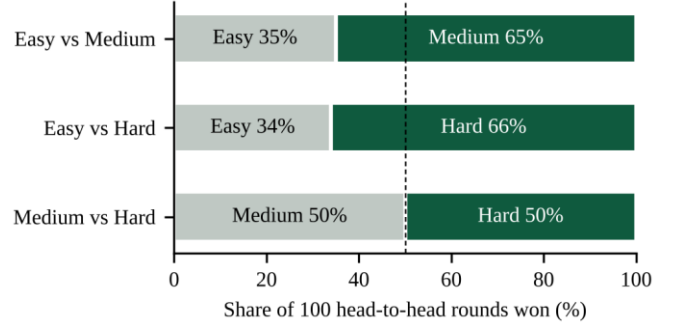


Fig. 4. Share of head-to-head rounds won by each agent level.

Medium and Hard tied at 50%. The Hard agent’s only addition is memory of dead cards, which enters through the connectivity term, and that term breaks ties only after the progress cost. Since the progress cost already decides most discards, the memory rarely changes a decision. This suggests that stronger play will require richer beliefs about hidden cards (for example, from opponents’ discards) or deeper search, rather than counting alone. For context, Saha *et al.* report over 60% for their best strategy against weaker profiles [22] and Mohan reports 53.9% for IRumAI against a search-based baseline [23]. The opponents differ, so the figures are not directly comparable, but they are of a similar magnitude.

C. Correctness and explanations

All 123 tests pass, including strict cases such as the rejection of K-A-2 runs, duplicate cards in a set, declarations without a pure sequence, the Ace-wild rule and reshuffling an exhausted stock. Because each recommendation is the result of an explicit evaluation, the coach always gives the reason behind it: the gain in structure, the change in loose points, the connected loose cards that would be lost, or the opponent who might benefit.

D. Limitations

The agents reason one ply ahead and model opponents only through recent pickups. Playing strength was measured between agents rather than against human players, and the educational benefit of the coach has not yet been tested in a user study.

VII. CONCLUSION AND FUTURE WORK

Smart Rummy shows that classical AI (backtracking, memoised dynamic programming, heuristic simulation and exact probability) is enough to build fast, testable and explainable agents for 13-card Indian Rummy. The utility-based agents beat a reflex baseline in about two-thirds of rounds, every decision takes well under a second, and each one comes with a human-readable explanation. Future work includes an information-set Monte Carlo agent [11] that uses

the dynamic program as its evaluation function, Bayesian opponent models, comparison with learned agents such as IRumAI [23], and a user study measuring how much the coach improves human play.

REFERENCES

- [1] CyberMedia Research and Unomer, “Cashing in: Inside India’s rummy gaming market,” CMR India, Gurugram, India, Nov. 2024. [Online]. Available: <https://cmrindia.com/>
- [2] State of Andhra Pradesh v. K. Satyanarayana and Others, AIR 1968 SC 825, Supreme Court of India, 1967.
- [3] Indian Institute of Technology Delhi, “Skill vs. chance in online rummy and poker,” study report, Sep. 2023, as reported by Storyboard18. [Online]. Available: <https://www.storyboard18.com/>
- [4] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018.
- [5] N. Brown and T. Sandholm, “Superhuman AI for multiplayer poker,” *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
- [6] A. Barredo Arrieta et al., “Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [7] J. Zhu, A. Liapis, S. Risi, R. Bidarra, and G. M. Youngblood, “Explainable AI for designers: A human-centered perspective on mixed-initiative co-creation,” in *Proc. IEEE Conf. Computational Intelligence and Games (CIG)*, Maastricht, The Netherlands, 2018.
- [8] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [10] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, “Understanding the success of perfect information Monte Carlo sampling in game tree search,” in *Proc. 24th AAAI Conf. Artificial Intelligence*, 2010, pp. 134–140.
- [11] P. I. Cowling, E. J. Powley, and D. Whitehouse, “Information set Monte Carlo tree search,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.
- [12] C. B. Browne et al., “A survey of Monte Carlo tree search methods,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [13] D. Billings, D. Papp, J. Schaeffer, and D. Szafron, “Opponent modeling in poker,” in *Proc. 15th National Conf. Artificial Intelligence (AAAI-98)*, Madison, WI, USA, 1998.
- [14] C. Kotnik and J. Kalita, “The significance of temporal-difference learning in self-play training: TD-rummy versus EVO-rummy,” in *Proc. 20th Int. Conf. Machine Learning (ICML)*, Washington, DC, USA, 2003.
- [15] S. T. Truong and T. W. Neller, “A data-driven approach for Gin Rummy hand evaluation,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15647–15654.
- [16] P. Goldman, C. R. Knutson, R. Mahtab, J. Maloney, J. B. Mueller, and R. G. Freedman, “Evaluating Gin Rummy hands using opponent modeling and myopic meld distance,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15510–15517.
- [17] M. Eicholtz, S. Moss, M. Traino, and C. Roberson, “Heisenbot: A rule-based game agent for Gin Rummy,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15489–15495.
- [18] S. Larkin, W. Collicott, and J. Hiebel, “Modeling expert knowledge in a heuristic-based Gin Rummy agent,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15577–15582.
- [19] D. Zha et al., “RLCard: A platform for reinforcement learning in card games,” in *Proc. 29th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Demos Track, 2020, pp. 5264–5266.
- [20] D. den Hertog and P. B. Hulshof, “Solving Rummikub problems by integer linear programming,” *The Computer Journal*, vol. 49, no. 6, pp. 665–669, 2006.
- [21] J. N. van Rijn, F. W. Takes, and J. K. Vis, “The complexity of Rummikub problems,” in *Proc. 27th Benelux Conf. Artificial Intelligence (BNAIC)*, 2015; arXiv:1604.07553.
- [22] P. Saha, A. Chakraborty, S. Sarkar, S. Maitra, D. Mukherjee, and T. Mukherjee, “Quantitative rule-based strategy modeling in classic Indian Rummy: A metric optimization approach,” arXiv:2601.00024, 2026.
- [23] V. Mohan, “IRumAI: Reinforcement learning for Indian Rummy,” arXiv:2606.21975, Jun. 2026.
- [24] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.